



Carnegie Mellon University
Master of
Software Engineering

Availability – timeout, retry, circuit breaker

LG Architecture Training Program

Paulo Merson

Availability tactics

- Detect faults
 - Monitor
 - Ping/echo
 - Heartbeat
- Recover from faults (or tolerate faults)
 - Active redundancy (hot spare)
 - Passive redundancy (warm spare)
 - Cold spare
 - Timeout
 - Retry
 - Circuit breaker



Timeout

- If a component needs to make a network call or access a resource, a timeout mechanism allows you to **stop waiting when you think the answer won't come**
- Libraries often have built-in timeouts for different actions, but you may need to configure or implement a timeout in some situations
- Well-placed timeouts provide fault isolation—a problem in some other service or device does not have to become your problem [Nygard18]





Retry

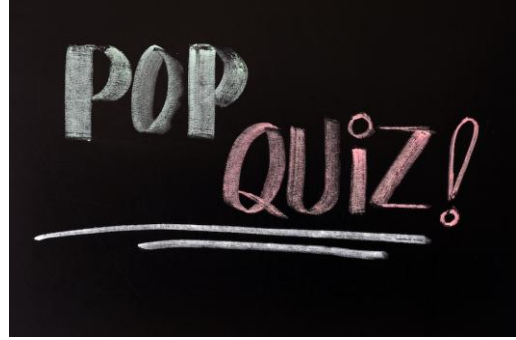
- Retrying an operation that failed due to an error or timeout may allow your application to **move forward in the face of transient problems**
- Drawbacks:
 - Increased delay in responding to the user
 - Increased memory utilization and resource consumption

Sometimes fail-fast is better than retry

Questions when designing a Retry solution

- How to distinguish transient errors from non-transient errors?
- Retry immediately after an error or retry after a delay?
- Fixed delay? How long? Incremental delay? Exponential backoff? Retry with jitter?
- How many retry attempts or total time to declare that the operation failed?
- Is the retried operation idempotent?
- If retrying a non-idempotent operation, is there a mechanism to enforce exactly-once execution, such as *Idempotent Receiver* [Joshi21], or message deduplication





Idempotent or non-idempotent?

- Remove an item from a collection **Idemp.**
- Add an item to a list **Non-idemp.**
- Add an item to a set **Idemp.**
- Multiply by two **Non-idemp.**
- Multiply by zero **Idemp.**
- Money withdrawal transaction **Non-idemp.**
- Lock a car using the remote key **Idemp.**



Circuit breaker

- Circuit breakers protect electrical circuits from current surges that can damage appliances and even cause fires



- The *circuit breaker pattern* protects the system from having service consumers waiting (and holding resources) for unresponsive services for a long time

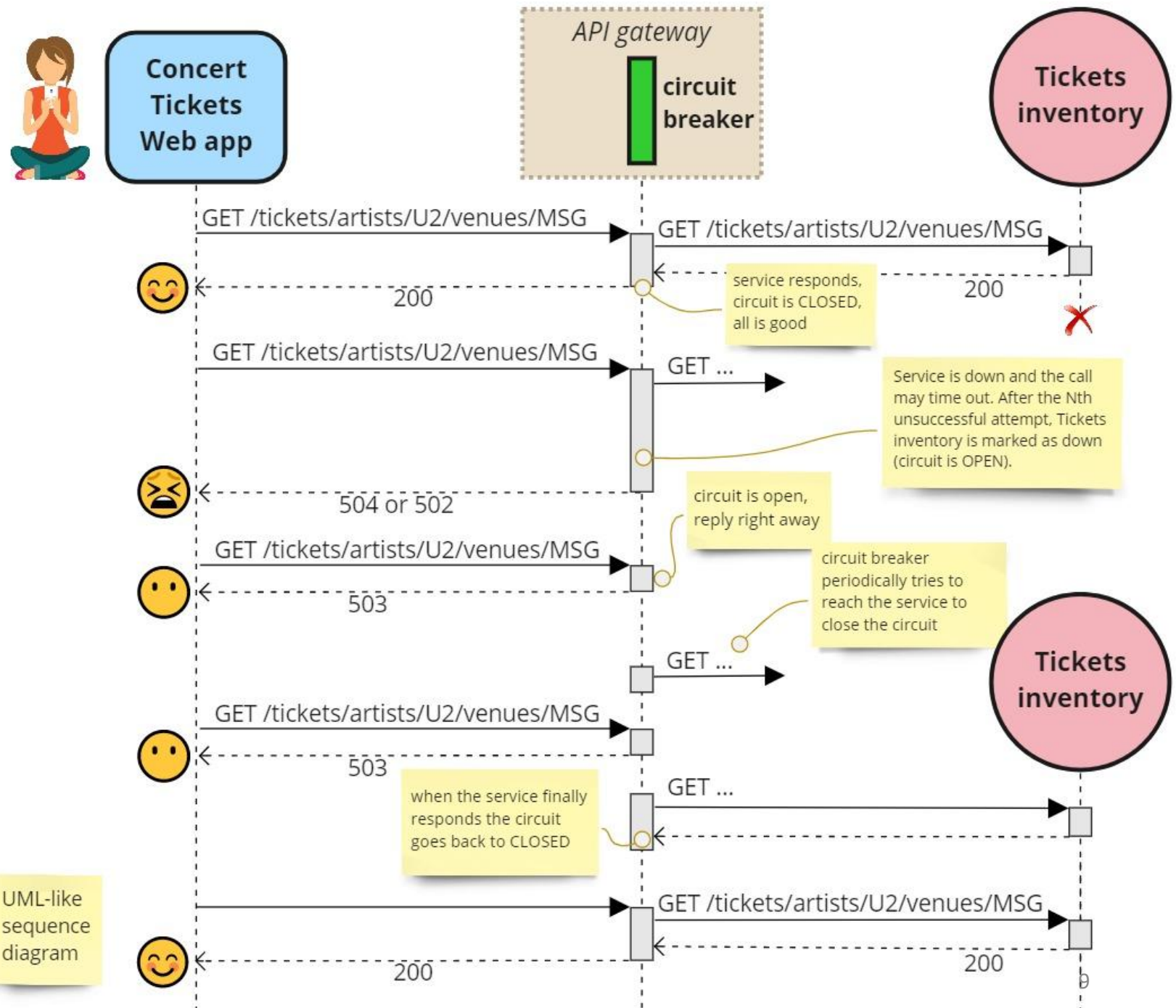
Circuit breaker for services

- The circuit breaker element intercepts calls to **synchronous services**
- If the service is responding, the circuit is **closed** 😊
- If the service is deemed unresponsive* 😞
 - the circuit becomes **open**
 - the breaker will immediately reject calls to that service 😬
- To move the circuit back to **closed** the breaker will periodically
 - either try to reach the service on its own to check if it's back
 - or allow a call to the service to go through to check if it responds

*Synchronous means
there's a request
followed by a response*

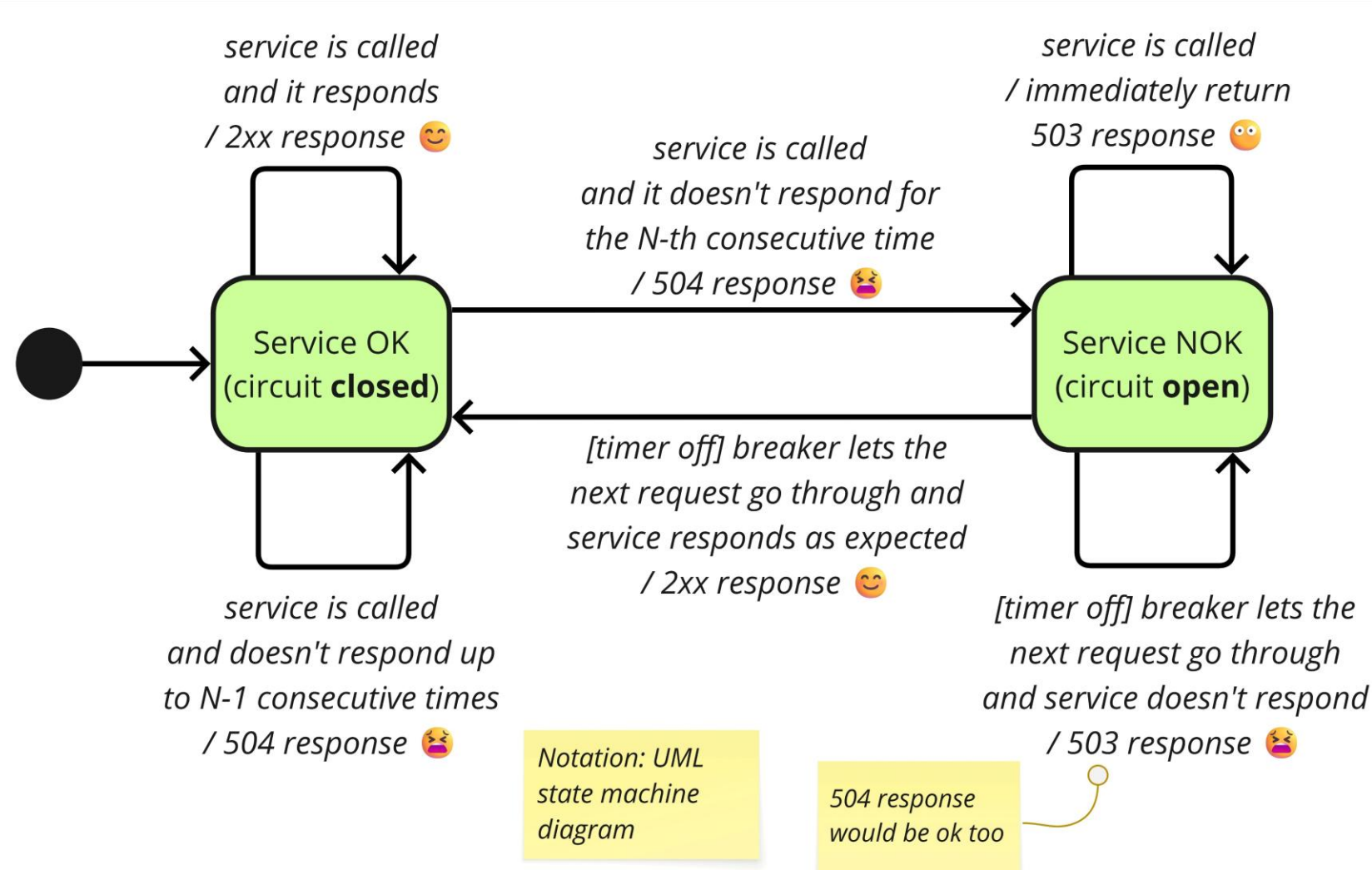
** Unresponsiveness is detected by N consecutive failures or a configurable failure rate threshold*

Example of a circuit breaker state transitions



*This is an example;
there are variations*

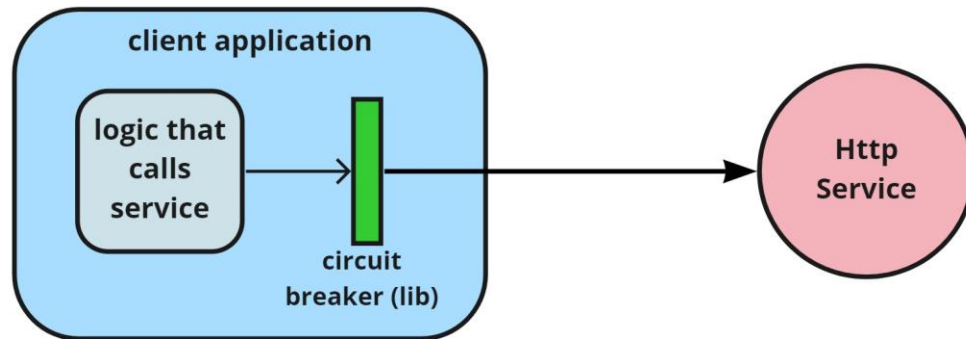
Circuit breaker state machine diagram



This is an example.
A variation of this
state machine is
when you have the
half-open state.

Circuit breaker implementation

- Select what service interactions might benefit from circuit breaking
- You can add circuit breaking to your client application at points where it calls services
- Circuit breaker libraries include Resilience4J, Polly, pybreaker, and Opossum



Other implementation options include an intermediary service, API Gateway or service mesh



More info in INygard181

Circuit breaker benefits

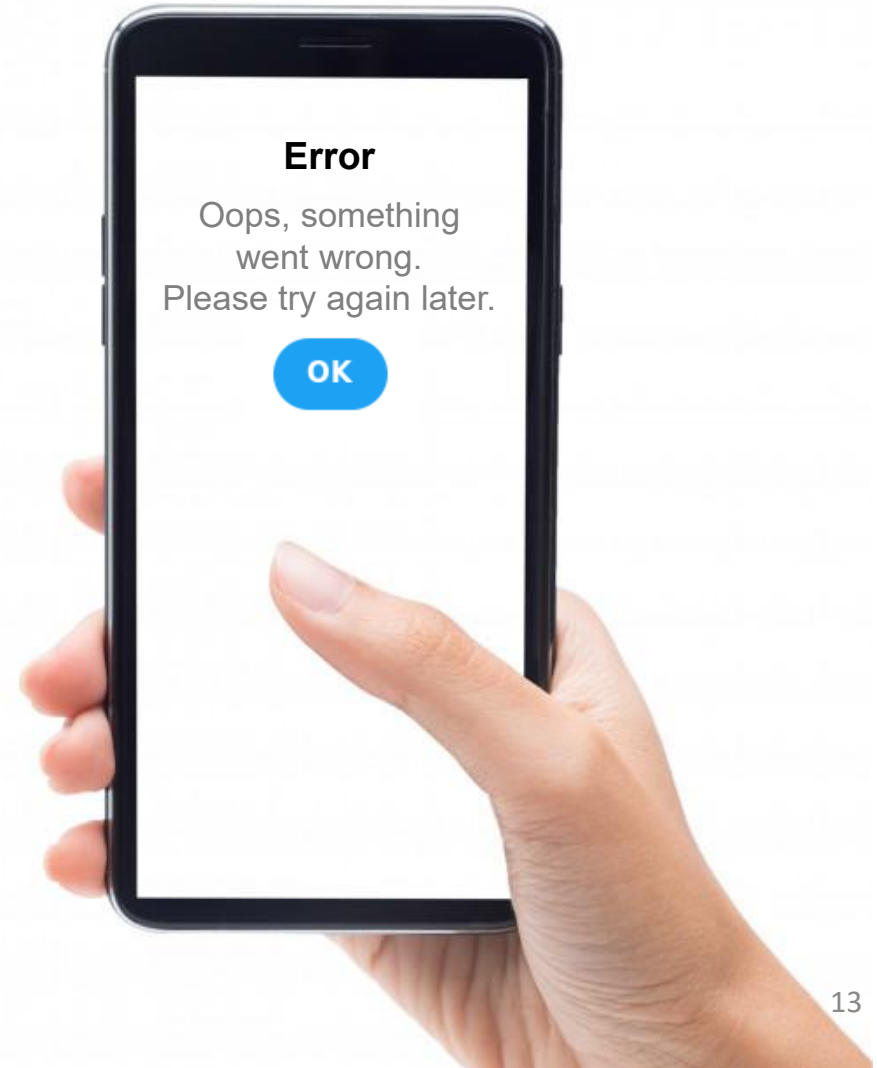
- Avoids that a service client holds resources for too long while waiting for an unresponsive service
 - Hence avoids that the service client's clients also block waiting
- Can also be used to activate a fallback mechanism for unresponsive services (e.g., respond with a cached value)
- Circuit state changes should be logged and monitored, and can provide indicators of problems with the services or infrastructure
- User experience (next slide)



Annoying



A better user experience



Circuit breaker challenges

- Added design complexity
- Governance burden in case the circuit breaker is independently deployed (one more component to deploy, monitor, etc.)
- Performance overhead as requests need to go through the circuit breaker
- Some requests that could be processed might be rejected while the circuit is open



*GenAI can help
you to understand
the tradeoffs!*





Questions?

Paulo Merson
pmerson@cmu.edu